Appendix

Team 2: Nathan Sabbara and Anthony Feaster II

This notebook is the code appendix for the report. The exploratory section is the initial EDA notebook. The later cells estimate the logistic model. An AI assistant drafted portions of the fitting and plotting code. We checked the sample and the fitted values against the output.

Initial Exploratory Data Analysis

Project: Predicting Successful NFL Third Down Conversions

This project examines NFLverse play-by-play data from the 2023-2026 seasons. The goal is to identify pre-play factors associated with successful third down conversions, including yards to go, field position, quarter, score differential, play type, formation and time remaining.

Data Source: [NFLverse Play-by-Play Data](https://github.com/nflverse/nflverse-data/releases/tag/pbp)

1. Data Loading and Initial Review

```
# Importing Libraries
from pathlib import Path
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Locating and setting the data folder
project_root = Path.cwd()
if project_root.name == "notebooks":
    project_root = project_root.parent

data_dir = project_root / "data" / "raw"

# Loading the data and creating the combined DataFrame
pbp_2023 = pd.read_csv(data_dir / "play_by_play_2023.csv", low_memory=False)
pbp_2024 = pd.read_csv(data_dir / "play_by_play_2024.csv", low_memory=False)
pbp_2025 = pd.read_csv(data_dir / "play_by_play_2025.csv", low_memory=False)
pbp_2026 = pd.read_csv(data_dir / "play_by_play_2026.csv", low_memory=False)

pbp = pd.concat([pbp_2023, pbp_2024, pbp_2025, pbp_2026], ignore_index=True)

#Commented out to avoid html submission issue
pbp.head(5)
```

```
   play_id          game_id  old_game_id home_team away_team season_type  \
0        1  2023_01_ARI_WAS   2023091007       WAS       ARI         REG
1       39  2023_01_ARI_WAS   2023091007       WAS       ARI         REG
2       55  2023_01_ARI_WAS   2023091007       WAS       ARI         REG
3       77  2023_01_ARI_WAS   2023091007       WAS       ARI         REG
4      102  2023_01_ARI_WAS   2023091007       WAS       ARI         REG

   week posteam posteam_type defteam  ... out_of_bounds  home_opening_kickoff  \
0     1     NaN          NaN     NaN  ...             0                     1
1     1     WAS         home     ARI  ...             0                     1
2     1     WAS         home     ARI  ...             0                     1
3     1     WAS         home     ARI  ...             0                     1
4     1     WAS         home     ARI  ...             0                     1

     qb_epa  xyac_epa  xyac_mean_yardage  xyac_median_yardage xyac_success  \
0  0.000000       NaN                NaN                  NaN          NaN
```

```
1  0.000000       NaN                NaN                 NaN          NaN
2 -0.336103       NaN                NaN                 NaN          NaN
3  0.703308  0.340652           3.328642                 1.0     0.996628
4  0.469799       NaN                NaN                 NaN          NaN

    xyac_fd     xpass    pass_oe
0       NaN       NaN        NaN
1       NaN       NaN        NaN
2       NaN  0.515058 -51.505846
3  0.583928  0.661106  33.889407
4       NaN  0.196065 -19.606467

[5 rows x 372 columns]
```

```
print(f"Total: {pbp.shape[0]:,} rows, {pbp.shape[1]:,} columns\n")
print(f"Dataset: {pbp.shape[0]:,} plays from 2023–2026 seasons\n")
pbp["season"].value_counts().sort_index()
```

```
Total: 156,084 rows, 372 columns

Dataset: 156,084 plays from 2023–2026 seasons


season
2023    49665
2024    49492
2025    48771
2026     8156
Name: count, dtype: int64
```

2. Reviewing the Complete Dataset

```
pbp.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 156084 entries, 0 to 156083
Columns: 372 entries, play_id to pass_oe
dtypes: float64(181), int64(39), object(152)
memory usage: 443.0+ MB
```

```
pbp.describe()
```

```
             play_id   old_game_id           week   yardline_100  \
count  156084.000000  1.560840e+05  156084.000000  144945.000000
mean     2186.581008  2.024291e+09       9.537070      47.756487
std      1274.076580  9.383605e+05       5.736511      23.932132
min         1.000000  2.023091e+09       1.000000       1.000000
25%      1092.000000  2.023122e+09       4.000000      30.000000
50%      2166.000000  2.024112e+09       9.000000      49.000000
75%      3263.000000  2.025102e+09      15.000000      68.000000
max      5519.000000  2.026093e+09      22.000000      99.000000

       quarter_seconds_remaining  half_seconds_remaining  \
count              156067.000000           156067.000000
mean                   408.920278              803.135583
std                    282.995402              560.986725
min                      0.000000                0.000000
25%                    143.000000              273.000000
50%                    388.000000              784.000000
75%                    654.000000             1285.000000
max                    900.000000             1800.000000

       game_seconds_remaining    quarter_end          drive             sp  \
count           156067.000000  156084.000000  154292.000000  156084.000000
mean              1704.052497       0.017651      11.318098       0.075447
std               1058.225076       0.131679       6.591909       0.264111
min                  0.000000       0.000000       1.000000       0.000000
25%                781.000000       0.000000       6.000000       0.000000
50%               1800.000000       0.000000      11.000000       0.000000
```

```
75%              2590.000000       0.000000      17.000000       0.000000
max              3600.000000       1.000000      34.000000       1.000000

       ...  out_of_bounds  home_opening_kickoff         qb_epa      xyac_epa  \
count  ...  156084.000000         156084.000000  154280.000000  52770.000000
mean   ...       0.085922              0.507547       0.006451      0.733447
std    ...       0.280249              0.499945       1.249021      0.528333
min    ...       0.000000              0.000000     -12.689102     -3.181688
25%    ...       0.000000              0.000000      -0.557776      0.325437
50%    ...       0.000000              1.000000       0.000000      0.608888
75%    ...       0.000000              1.000000       0.531366      1.007500
max    ...       1.000000              1.000000       8.882384      9.585749

       xyac_mean_yardage  xyac_median_yardage  xyac_success       xyac_fd  \
count       52770.000000         52770.000000  52770.000000  52770.000000
mean            5.527103             3.702274      0.775842      0.578875
std             2.564078             2.630531      0.254090      0.356944
min             0.442673             0.000000      0.026981      0.004220
25%             3.756694             2.000000      0.555827      0.242725
50%             4.621278             3.000000      0.983180      0.474688
75%             6.920977             5.000000      1.000000      0.999328
max            23.952005            36.000000      1.000000      1.000000

               xpass        pass_oe
count  118896.000000  115501.000000
mean        0.628758      -1.418037
std         0.240985      42.391991
min         0.013008     -99.538189
25%         0.456023     -42.579734
50%         0.601380       3.849697
75%         0.859060      32.931900
max         0.997794      97.848164

[8 rows x 220 columns]
```

```
pbp.columns
```

```
Index(['play_id', 'game_id', 'old_game_id', 'home_team', 'away_team',
       'season_type', 'week', 'posteam', 'posteam_type', 'defteam',
       ...
       'out_of_bounds', 'home_opening_kickoff', 'qb_epa', 'xyac_epa',
       'xyac_mean_yardage', 'xyac_median_yardage', 'xyac_success', 'xyac_fd',
       'xpass', 'pass_oe'],
      dtype='object', length=372)
```

3. Initial Data Quality Review

```
#Looking for Missing Values
pbp.isnull().sum()
```

```
play_id                     0
game_id                     0
old_game_id                 0
home_team                   0
away_team                   0
                          ...
xyac_median_yardage    103314
xyac_success           103314
xyac_fd                103314
xpass                   37188
pass_oe                 40583
Length: 372, dtype: int64
```

```
# Checking for Duplicate Values
print(f"Number of Duplicate Values: {pbp.duplicated().sum()}")
```

```
Number of Duplicate Values: 0
```

4. Defining the Plays Included in the Analysis

```
# Creating a new dataframe of only 3rd Down Plays
third_down = pbp[pbp["down"] == 3].copy()

print(f"Dataset: {third_down.shape[0]:,} third down plays from 2023–2026 seasons\n")
```

```
Dataset: 25,422 third down plays from 2023–2026 seasons
```

```
# Reviewing the types of third down plays
third_down['play_type'].value_counts(dropna=False)
```

```
play_type
pass          17180
run            5765
no_play        2162
qb_kneel        190
field_goal      100
qb_spike         25
Name: count, dtype: int64
```

```
# Reviewing the target variable
third_down["third_down_converted"].value_counts(dropna=False).rename(index={0: "Failed", 1:
"Successful"})
```

```
third_down_converted
Failed        16257
Successful     9124
NaN              41
Name: count, dtype: int64
```

```
# Identifying third-down records with missing conversion outcomes
missing_outcomes = third_down[third_down["third_down_converted"].isna()]

# Checking which play types have missing outcomes
missing_outcomes["play_type"].value_counts(dropna=False)
```

```
play_type
no_play    41
Name: count, dtype: int64
```

41 third down plays had missing conversion outcomes `NaN`. All 41 records were classified as `no_play` and described penalty events. Only regular-season run and pass plays were retained for analysis, so these records were not included. No missing conversion outcomes remained among the selected plays.

```
# Reviewing regular season vs playoffs
third_down['season_type'].value_counts(dropna=False)
```

```
season_type
REG     24344
POST     1078
Name: count, dtype: int64
```

```
# Keeping regular-season third-down run and pass attempts with a recorded conversion outcome
third_down_analysis = third_down[
    (third_down['season_type'] == 'REG') &
    (third_down['play_type'].isin (['run','pass'])) &
    (third_down['third_down_converted'].notna())
].copy()

print(f"Dataset: {third_down_analysis.shape[0]:,} third down plays from 2023–2026 seasons\n")
```

```
Dataset: 21,948 third down plays from 2023–2026 seasons
```

5. Selecting the Analysis Variables

```
analysis_columns = [
    "game_id",
    "play_id",
    "season",
    "week",
    "posteam",
    "defteam",
    "qtr",
    "game_seconds_remaining",
    "yardline_100",
    "ydstogo",
    "play_type",
    "third_down_converted",
    "posteam_timeouts_remaining",
    "defteam_timeouts_remaining",
    "score_differential",
    "shotgun"
]
third_down_plays = third_down_analysis[analysis_columns].copy()
third_down_plays.shape
```

```
(21948, 16)
```

```
# Looking for missing values in the selected analysis variables

missing_values = pd.DataFrame({
    "missing_count": third_down_plays.isna().sum(),
    "missing_percentage": third_down_plays.isna().mean() * 100
})

missing_values = missing_values.sort_values(by="missing_percentage", ascending=False)
missing_values
```

	missing_count	missing_percentage
game_id	0	0.0
play_id	0	0.0
season	0	0.0
week	0	0.0
posteam	0	0.0
defteam	0	0.0
qtr	0	0.0
game_seconds_remaining	0	0.0
yardline_100	0	0.0
ydstogo	0	0.0
play_type	0	0.0
third_down_converted	0	0.0
posteam_timeouts_remaining	0	0.0
defteam_timeouts_remaining	0	0.0
score_differential	0	0.0
shotgun	0	0.0

```
print("Complete Duplicate Rows:", third_down_plays.duplicated().sum())
print("Duplicate Rows by Play ID:", third_down_plays.duplicated(subset=["game_id",
"play_id"]).sum())
```

```
Complete Duplicate Rows: 0
Duplicate Rows by Play ID: 0
```

There are 21,948 plays in the dataset after filtering down to regular season third-down run and pass attempts with a recorded conversion outcome. 16 selected variables (columns) describe the game situation, play type, field position, time remaining, and conversion outcome. There were no missing values and no duplicate plays found.

6. Third-Down Conversion Analysis

Conversion Summary Function

```
def summarize_conversions(data, group_column):
    summary = (
        data.groupby(group_column, as_index=False).agg(
            number_of_plays=(
                "third_down_converted", "count"
            ),
            successful_conversions=(
                "third_down_converted", "sum"
            ),
            conversion_rate=(
                "third_down_converted", "mean"
            )
        )
    )
    summary["conversion_rate"] = (summary["conversion_rate"] * 100).round(2)

    return summary
```

6.1 Overall Conversion Results

```
conversion_summary = (third_down_plays["third_down_converted"].value_counts().reindex([0, 1],
fill_value=0).rename_axis("conversion").reset_index(name="count"))

# Adding Descriptive Labels
conversion_summary["outcome"] = conversion_summary["conversion"].map({0: "Failed", 1:
"Successful"})

# Calculating the proportion and percentage of third down plays
conversion_summary["rate"] = (conversion_summary["count"] / conversion_summary["count"].sum())

conversion_summary["rate_percentage"] = (conversion_summary["rate"] * 100).round(2).map("
{:.2f}%".format)

# Renaming Column Names to be more descriptive
conversion_summary = conversion_summary[["outcome", "count",
"rate_percentage"]].rename(columns={
    "outcome": "Conversion Outcome",
    "count": "Number of Plays",
    "rate_percentage": "Percentage of Plays"
})

conversion_summary.style.format({
    "Number of Plays": "{:,.0f}",
})
```

	Conversion Outcome	Number of Plays	Percentage of Plays
0	Failed	13,236	60.31%
1	Successful	8,712	39.69%

Out of the 21,948 third down plays, 8,712 were successful which resulted in a conversion rate of 39.69%, while 13,236 were unsuccessful which resulted in a failed rate of 60.31%. This means that the majority of third down plays were unsuccessful, highlighting the challenges teams face when converting on third down situations.

6.2 Conversion Rate by Season

```
# Comparing Conversion Rates by Seasons
conversion_by_season = (third_down_plays.groupby("season", as_index=False)
.agg(
    number_of_plays = ("third_down_converted", "count"),
    successful_conversions = ("third_down_converted", "sum"),
    conversion_rate = ("third_down_converted", "mean")
))

# Converting Rate to Percentage
conversion_by_season["conversion_rate"] = (conversion_by_season["conversion_rate"] *
100).round(2).map("{:.2f}%".format)

# Renaming Columns
conversion_by_season = conversion_by_season.rename(columns={
    "season": "Season",
    "number_of_plays": "Number of Plays",
    "successful_conversions": "Successful Conversions",
    "conversion_rate": "Conversion Rate (%)"
})

conversion_by_season.style.format({
    "Number of Plays": "{:,.0f}",
    "Successful Conversions": "{:,.0f}",
})
```

	Season	Number of Plays	Successful Conversions	Conversion Rate (%)
0	2023	7,140	2,788	39.05%
1	2024	6,828	2,728	39.95%
2	2025	6,801	2,710	39.85%
3	2026	1,179	486	41.22%

```
sns.set_theme(style="whitegrid")

season_plot = (third_down_plays.groupby("season", as_index=False)
["third_down_converted"].mean())

season_plot["conversion_rate"] = season_plot["third_down_converted"] * 100

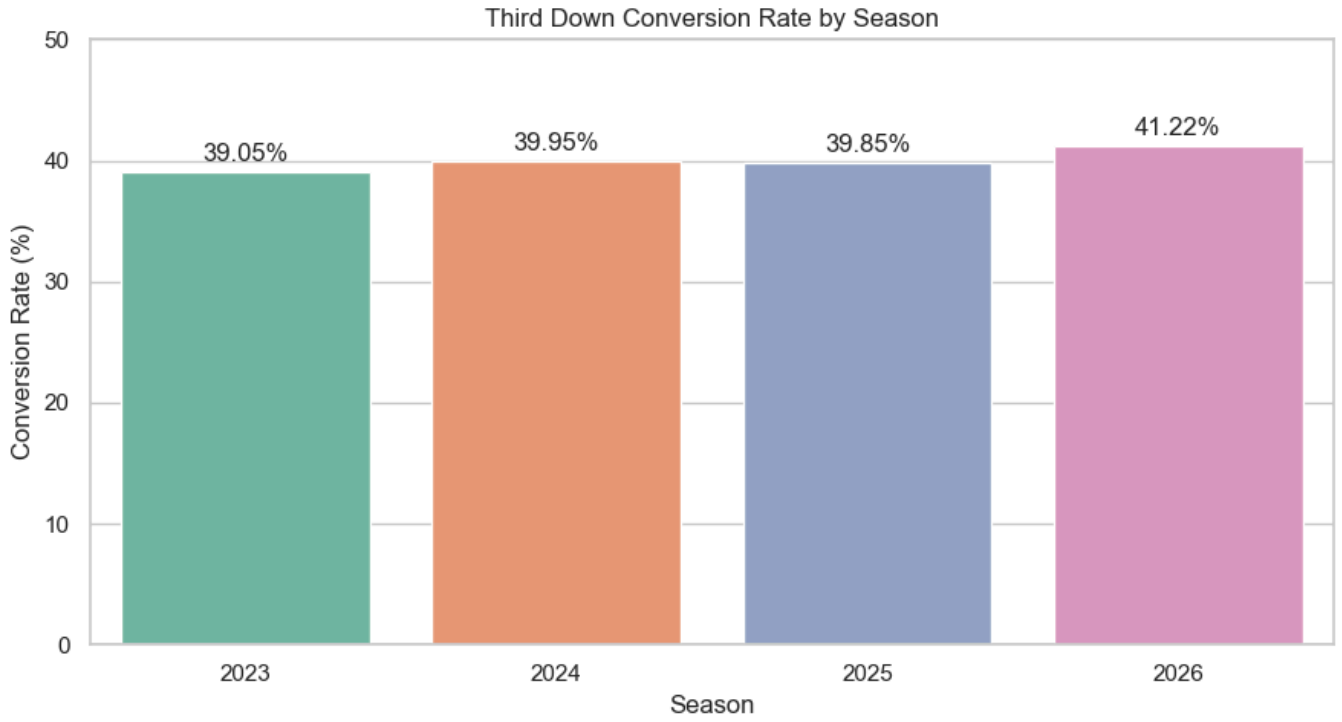
fig, ax = plt.subplots(figsize=(9, 5))

sns.barplot(
    data=season_plot,
    x="season",
    y="conversion_rate",
    hue="season",
    palette="Set2",
    dodge=False,
    legend=False,
    ax=ax
)

for container in ax.containers:
    ax.bar_label(container, fmt="%.2f%%", padding=3)

ax.set_title("Third Down Conversion Rate by Season")
ax.set_xlabel("Season")
ax.set_ylabel("Conversion Rate (%)")
ax.set_ylim(0, 50)
ax.grid(axis="x", visible=False)
```

```
plt.tight_layout()
plt.show()
```



Third down conversion rates remained relatively consistent across the four seasons, ranging from 39.05% to 41.22%. The partial 2026 season had the highest conversion rate, but it contained fewer plays than the completed seasons. Therefore, comparisons with previous years should be interpreted cautiously.

6.3 Conversion Rate by Play Type

```
conversion_by_play_type = summarize_conversions(third_down_plays, "play_type")

conversion_by_play_type_display = conversion_by_play_type.rename(
    columns={
        "play_type": "Play Type",
        "number_of_plays": "Number of Plays",
        "successful_conversions": "Successful Conversions",
        "conversion_rate": "Conversion Rate (%)"
    }
).copy()

conversion_by_play_type_display['Play Type'] = (conversion_by_play_type_display['Play
Type'].str.title())
conversion_by_play_type_display['Conversion Rate (%)'] =
(conversion_by_play_type_display['Conversion Rate (%)'].map("{:.2f}%".format))

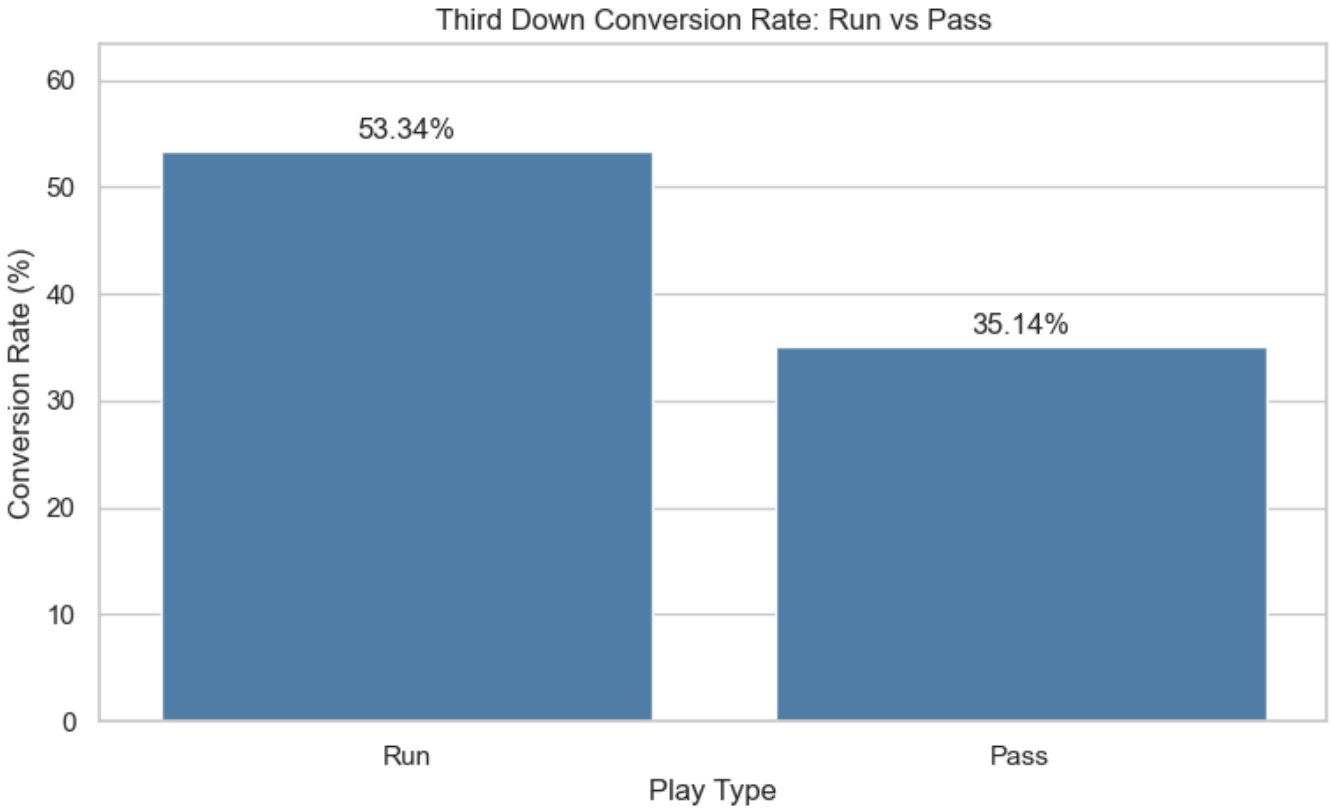
conversion_by_play_type_display.style.format({
    "Number of Plays": "{:,.0f}",
    "Successful Conversions": "{:,.0f}"
})
```

	Play Type	Number of Plays	Successful Conversions	Conversion Rate (%)
0	Pass	16,455	5,782	35.14%
1	Run	5,493	2,930	53.34%

```
plt.figure(figsize=(9, 5))

ax = sns.barplot(
    data=conversion_by_play_type,
    x = conversion_by_play_type["play_type"].str.title(),
    y = "conversion_rate",
    order = ["Run", "Pass"],
    color = "steelblue"
)

plt.title("Third Down Conversion Rate: Run vs Pass")
plt.xlabel("Play Type")
plt.ylabel("Conversion Rate (%)")
plt.ylim(0, conversion_by_play_type["conversion_rate"].max() + 10)
for container in ax.containers:
    ax.bar_label(container, fmt="%.2f%%", padding=3)
plt.show()
```



The data reveals that running plays had a higher third-down conversion rate than passing plays. Running plays were converted 53.34% of the time compared to 35.14% for passing plays. This doesn't mean that running caused a higher degree of conversions; teams may be more likely to run when fewer yards are needed for a first down.

6.4 Conversion Rate by Yards to Go

```
third_down_plays["yards_to_go_group"] = pd.cut(third_down_plays["ydstogo"],
    bins=[0, 2, 5, 10, np.inf],
    labels=[
        "1–2 Yards",
        "3–5 Yards",
        "6–10 Yards",
        "11+ Yards"
    ]
```

```
)

conversion_by_distance = summarize_conversions(third_down_plays, "yards_to_go_group")

conversion_by_distance_display = conversion_by_distance.rename(
    columns={
        "yards_to_go_group": "Yards to Go",
        "number_of_plays": "Number of Plays",
        "successful_conversions": "Successful Conversions",
        "conversion_rate": "Conversion Rate (%)"
    }
)

# Display successful conversion counts as whole numbers
conversion_by_distance_display["Successful Conversions"] = (
    conversion_by_distance_display["Successful Conversions"].astype(int)
)
conversion_by_distance_display['Conversion Rate (%)'] =
(conversion_by_distance_display['Conversion Rate (%)'].map("{:.2f}%".format))


conversion_by_distance_display.style.format({
    "Number of Plays": "{:,.0f}",
    "Successful Conversions": "{:,.0f}"
})
```

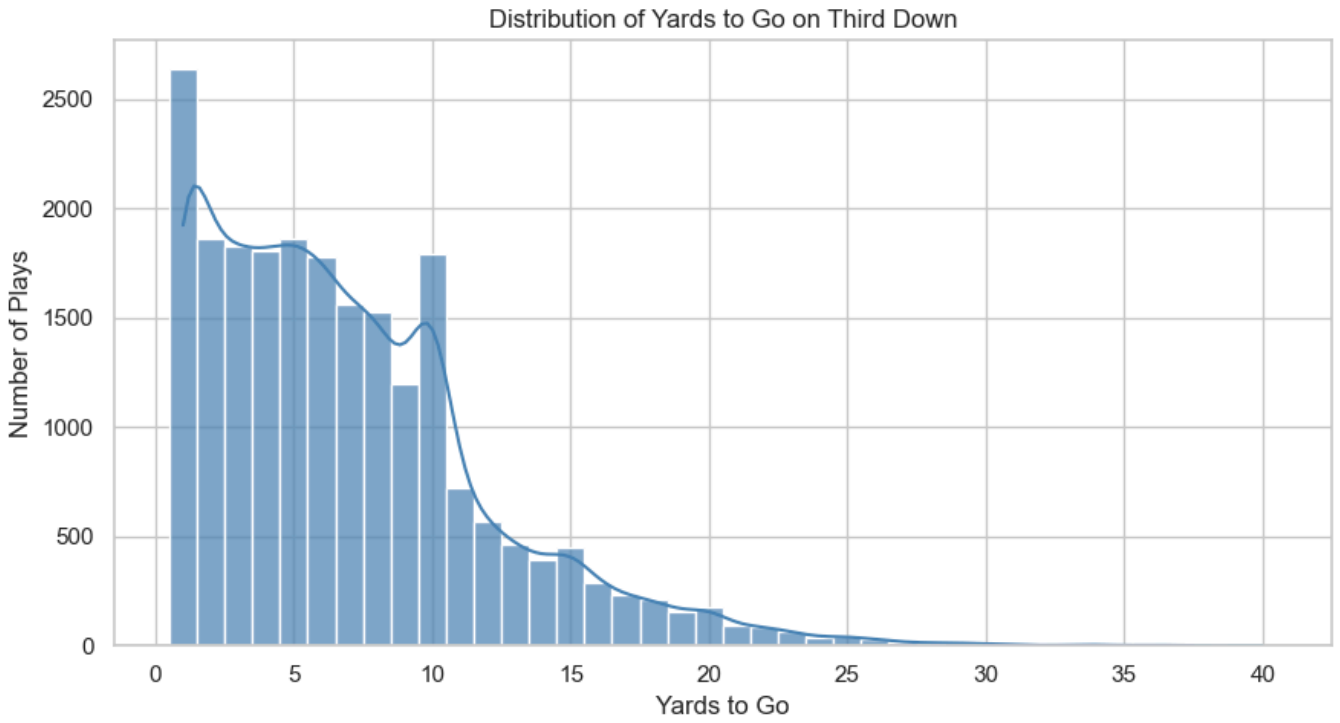
```
/var/folders/7j/tm7h019108s7yxqq8_pr_8z00000gn/T/ipykernel_54391/2302337863.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  data.groupby(group_column, as_index=False).agg(
```

	Yards to Go	Number of Plays	Successful Conversions	Conversion Rate (%)
0	1-2 Yards	4,498	2,851	63.38%
1	3-5 Yards	5,497	2,595	47.21%
2	6-10 Yards	7,866	2,650	33.69%
3	11+ Yards	4,087	616	15.07%

```
plt.figure(figsize=(9, 5))

sns.histplot(
    data=third_down_plays,
    x="ydstogo",
    discrete=True,
    kde=True,
    color="steelblue",
    alpha=0.7
)

plt.title("Distribution of Yards to Go on Third Down")
plt.xlabel("Yards to Go")
plt.ylabel("Number of Plays")
plt.tight_layout()
plt.show()
```

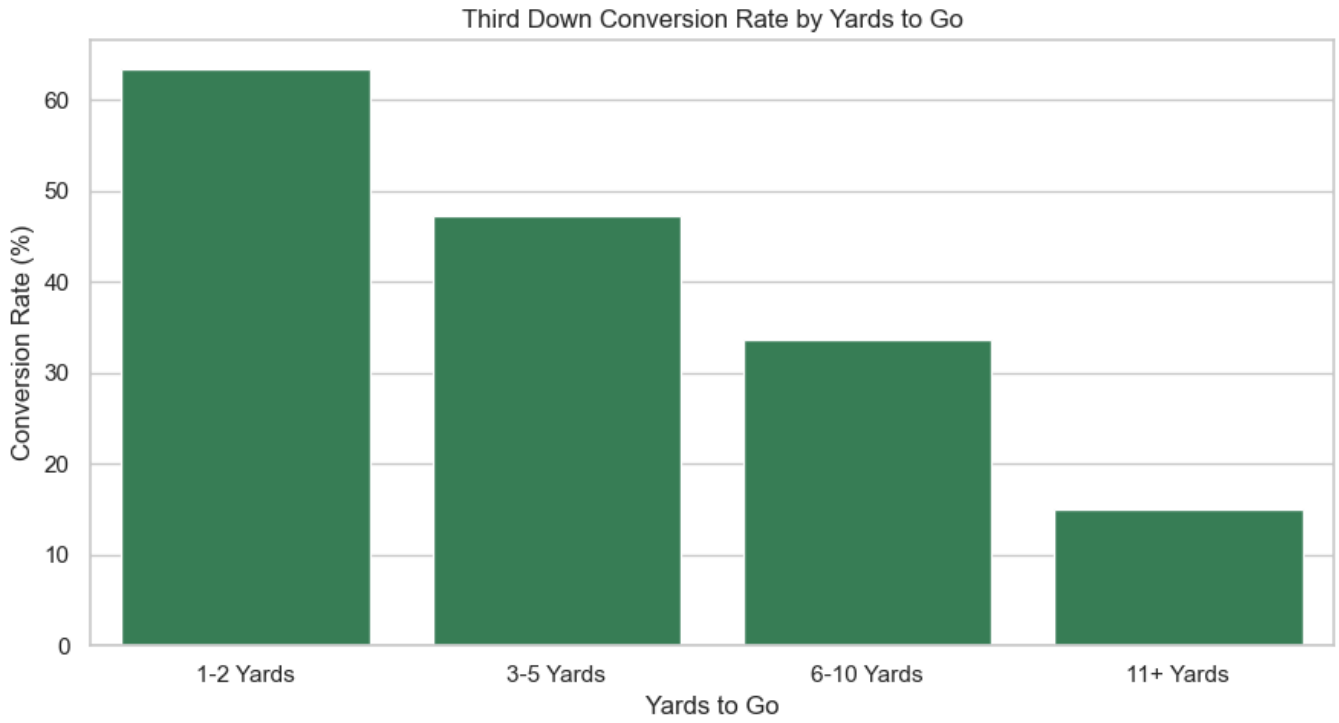


This histogram shows the distribution of yards to go on third downs. Each bar represents the number of plays at a particular distance. The histogram illustrates that majority of third down plays were 10 yards or less.

```
plt.figure(figsize=(9, 5))

sns.barplot(
    data = conversion_by_distance,
    x = conversion_by_distance["yards_to_go_group"].str.title(),
    y = "conversion_rate",
    color = "seagreen"
)

plt.title("Third Down Conversion Rate by Yards to Go")
plt.xlabel("Yards to Go")
plt.ylabel("Conversion Rate (%)")
plt.tight_layout()
plt.show()
```



As the distance for a first down increased, the successful conversion rate decreased. Plays needing 1–2 yards for a first down were converted 63.38% of the time. However, plays needing 11+ yards only converted 15.07% of the time. This suggests that yards to go has a strong relationship with successful third down conversions.

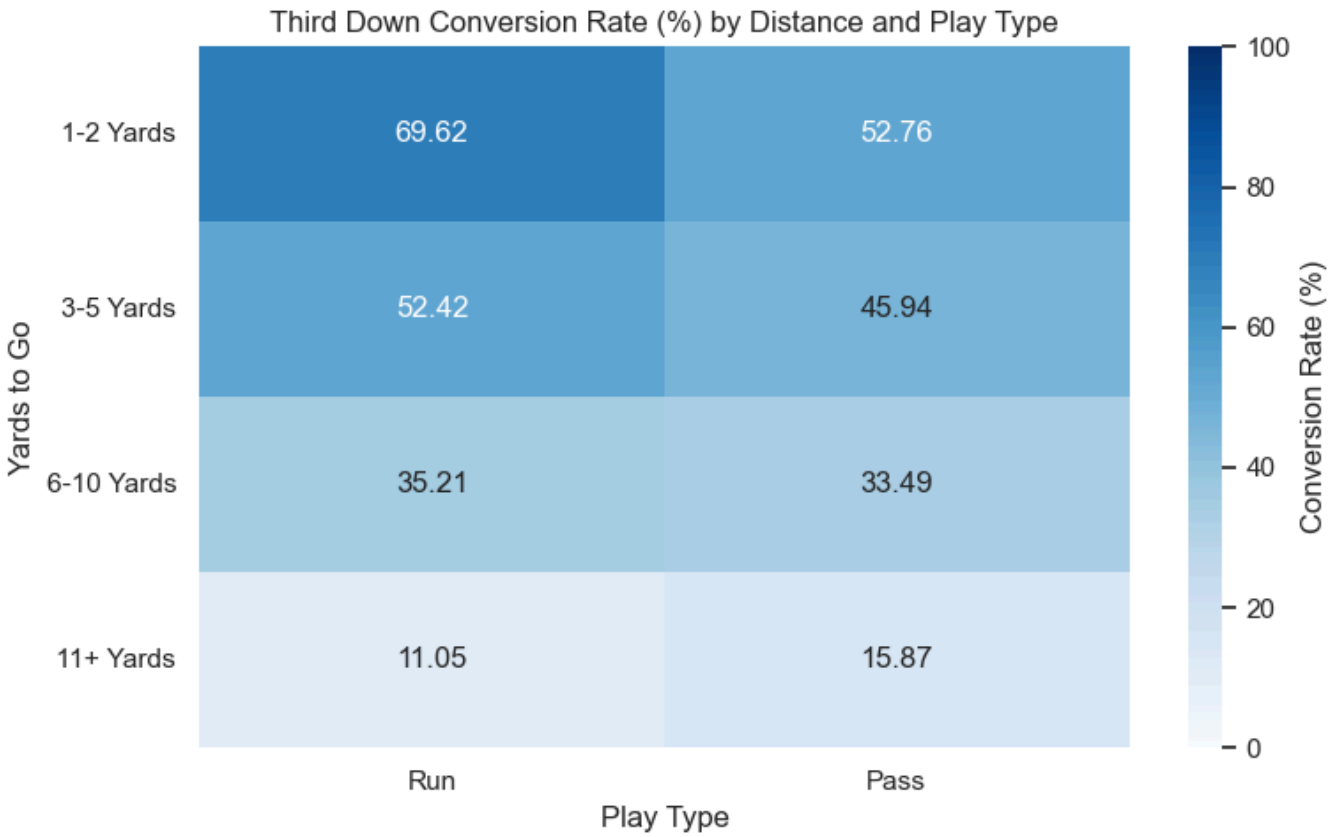
6.5 Distance and Play Type

```
conversion_heatmap = third_down_plays.pivot_table(
    values="third_down_converted",
    index="yards_to_go_group",
    columns="play_type",
    aggfunc="mean",
    observed=False
) * 100

conversion_heatmap = conversion_heatmap.reindex(columns=["run", "pass"])
conversion_heatmap = conversion_heatmap.rename(columns={"run": "Run", "pass": "Pass"})

plt.figure(figsize=(8, 5))
sns.heatmap(
    conversion_heatmap,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    vmin=0,
    vmax=100,
    cbar_kws={"label": "Conversion Rate (%)"}
)

plt.title("Third Down Conversion Rate (%) by Distance and Play Type")
plt.xlabel("Play Type")
plt.ylabel("Yards to Go")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```



This heatmap compares run and pass conversion rates within the same yards to go categories. It shows how teams were more successful at converting third downs with running plays when the yards to go were 5 and under. When first downs required 11+ yards, teams were more successful with passing plays.

6.6 Conversion Rate by Quarter

```
conversion_by_quarter = summarize_conversions(third_down_plays, "qtr")

conversion_by_quarter_display = conversion_by_quarter.rename(
    columns={
        "qtr": "Quarter",
        "number_of_plays": "Number of Plays",
        "successful_conversions": "Successful Conversions",
        "conversion_rate": "Conversion Rate (%)"
    }
)

# Display successful conversion counts as whole numbers
conversion_by_quarter_display["Successful Conversions"] = (
    conversion_by_quarter_display["Successful Conversions"].astype(int)
)

# Display quarters as whole numbers
conversion_by_quarter_display["Quarter"] = (
    conversion_by_quarter_display["Quarter"].astype(int)
)

# Display conversion rate with a percentage sign
conversion_by_quarter_display['Conversion Rate (%)'] =
(conversion_by_quarter_display['Conversion Rate (%)'].map("{:.2f}%".format))

# Display quarter 5 as Overtime
```

```
conversion_by_quarter_display["Quarter"] = (
    conversion_by_quarter_display["Quarter"].replace({
        1: "1st",
        2: "2nd",
        3: "3rd",
        4: "4th",
        5: "Overtime"
        })
)

conversion_by_quarter_display.style.format({
    "Number of Plays": "{:,.0f}",
    "Successful Conversions": "{:,.0f}"
})
```

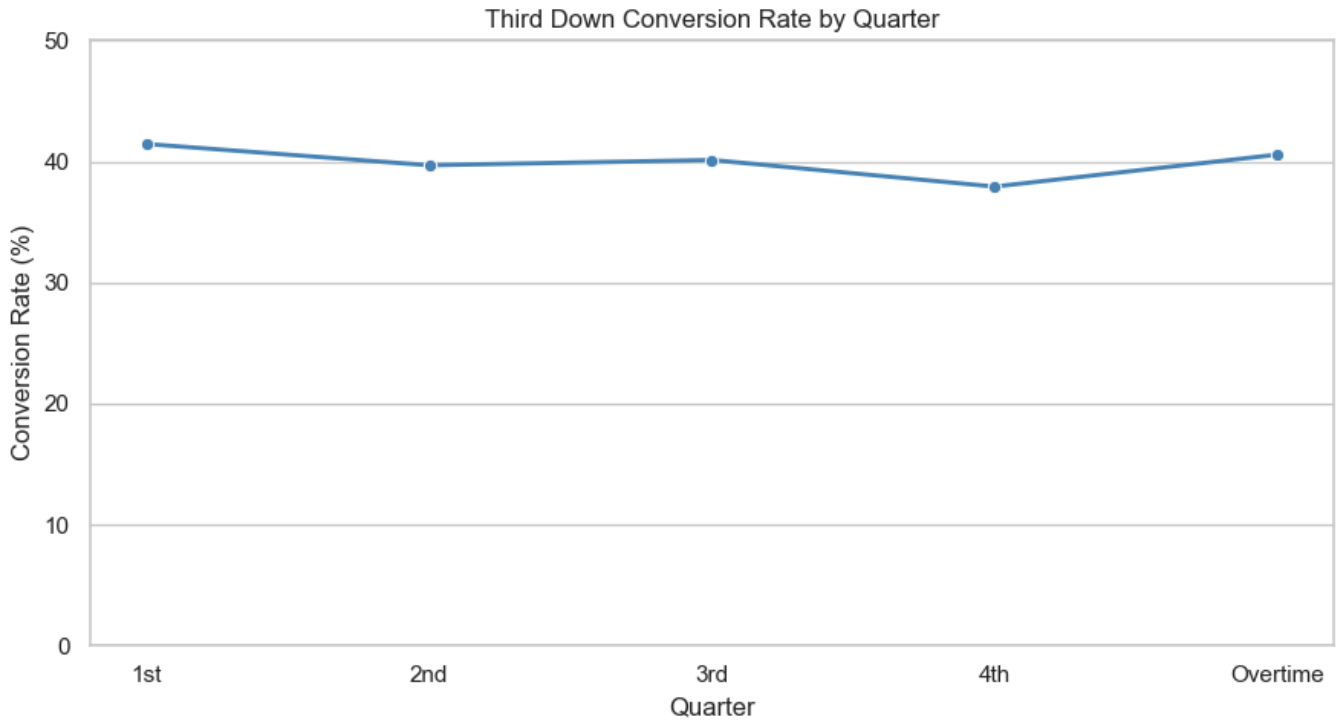
	Quarter	Number of Plays	Successful Conversions	Conversion Rate (%)
0	1st	4,890	2,027	41.45%
1	2nd	6,054	2,403	39.69%
2	3rd	4,797	1,925	40.13%
3	4th	6,101	2,314	37.93%
4	Overtime	106	43	40.57%

```
fig, ax = plt.subplots(figsize=(9, 5))

sns.lineplot(
    data=conversion_by_quarter,
    x="qtr",
    y="conversion_rate",
    marker="o",
    linewidth=2,
    color="steelblue",
    sort=False,
    ax=ax
)


ax.set_title("Third Down Conversion Rate by Quarter")
ax.set_xlabel("Quarter")
ax.set_xticks(
    [1, 2, 3, 4, 5],
    labels=["1st", "2nd", "3rd", "4th", "Overtime"]
)
ax.set_ylabel("Conversion Rate (%)")
ax.set_ylim(0, 50)
ax.grid(axis="x", visible=False)

plt.tight_layout()
plt.show()
```



Conversion rates were relatively similar across the four regulation quarters. The first quarter had the highest conversion rate at 41.45%, compared with 37.93% in the fourth quarter. Overtime had a conversion rate of 40.57%, but contained only 106 plays.

6.7 Conversion Rate by Field Position

```
third_down_plays["field_position_group"] = pd.cut(
    third_down_plays["yardline_100"],
    bins=[0, 20, 50, 79, 100],
    labels=[
        "Opponent 1–20",
        "Opponent 21–50",
        "Own 21–50",
        "Own 1–20"
    ],
    include_lowest=True
)

conversion_by_field_position = summarize_conversions(third_down_plays, "field_position_group")

conversion_by_field_position_display = conversion_by_field_position.rename(
    columns={
        "field_position_group": "Field Position",
        "number_of_plays": "Number of Plays",
        "successful_conversions": "Successful Conversions",
        "conversion_rate": "Conversion Rate (%)"
    }
)

conversion_by_field_position_display['Conversion Rate (%)'] =
(conversion_by_field_position_display['Conversion Rate (%)'].map("{:.2f}%".format))

conversion_by_field_position_display.style.format({
    "Number of Plays": "{:,.0f}",
```

```
    "Successful Conversions": "{:,.0f}"
})
```

```
/var/folders/7j/tm7h019108s7yxqq8_pr_8z00000gn/T/ipykernel_54391/2302337863.py:3: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
  data.groupby(group_column, as_index=False).agg(
```

	Field Position	Number of Plays	Successful Conversions	Conversion Rate (%)
0	Opponent 1-20	3,678	1,470	39.97%
1	Opponent 21-50	7,006	2,884	41.16%
2	Own 21-50	9,431	3,791	40.20%
3	Own 1-20	1,833	567	30.93%

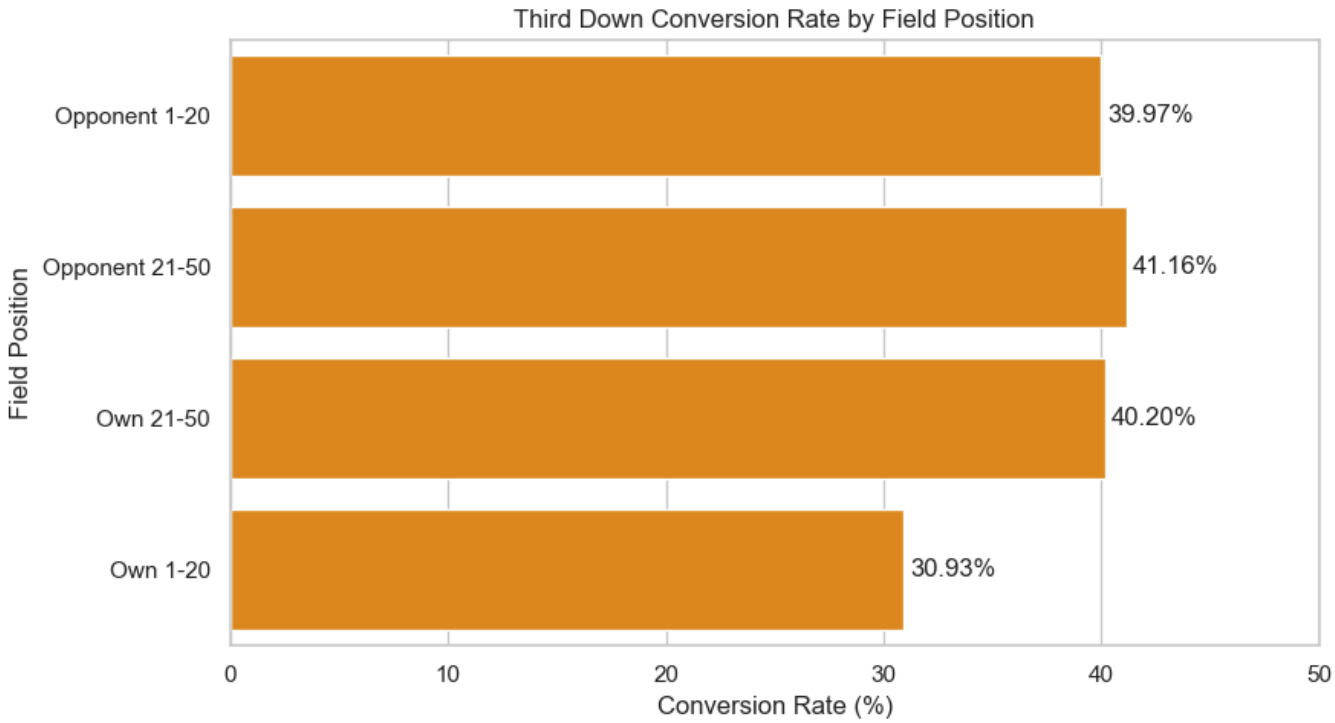
```
fig, ax = plt.subplots(figsize=(9, 5))

sns.barplot(
    data=conversion_by_field_position,
    y="field_position_group",
    x="conversion_rate",
    color="darkorange",
    ax=ax
)

for container in ax.containers:
    ax.bar_label(
        container,
        fmt="%.2f%%",
        padding=3
    )

ax.set_title("Third Down Conversion Rate by Field Position")
ax.set_xlabel("Conversion Rate (%)")
ax.set_ylabel("Field Position")
ax.set_xlim(0, 50)
ax.grid(axis="y", visible=False)

plt.tight_layout()
plt.show()
```



Conversion rates were highest between the opponent's 21–50 yard lines (41.16%) and lowest between the offense's own 1–20 yard lines (30.93%).

6.8 Conversion Rate by Score Situation

```
third_down_plays["score_situation"] = pd.cut(
    third_down_plays["score_differential"],
    bins = [-np.inf, -9, -1, 0, 8, np.inf],
    labels = [
        "Trailing by 9+",
        "Trailing by 1–8",
        "Tied",
        "Leading by 1–8",
        "Leading by 9+"
    ]
)

conversion_by_score = summarize_conversions(third_down_plays, "score_situation")

conversion_by_score_display = conversion_by_score.rename(
    columns={
        "score_situation": "Score Situation",
        "number_of_plays": "Number of Plays",
        "successful_conversions": "Successful Conversions",
        "conversion_rate": "Conversion Rate (%)"
    }
)

conversion_by_score_display['Conversion Rate (%)'] = (conversion_by_score_display['Conversion
Rate (%)'].map("{:.2f}%".format))

conversion_by_score_display.style.format({
    "Number of Plays": "{:,.0f}",
    "Successful Conversions": "{:,.0f}"
})
```

```
/var/folders/7j/tm7h019108s7yxqq8_pr_8z00000gn/T/ipykernel_54391/2302337863.py:3: FutureWarning: The
default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass
observed=False to retain current behavior or observed=True to adopt the future default and silence this
warning.
  data.groupby(group_column, as_index=False).agg(
```

	Score Situation	Number of Plays	Successful Conversions	Conversion Rate (%)
0	Trailing by 9+	4,228	1,622	38.36%
1	Trailing by 1-8	5,919	2,346	39.64%
2	Tied	4,333	1,782	41.13%
3	Leading by 1-8	4,571	1,795	39.27%
4	Leading by 9+	2,897	1,167	40.28%

The conversion rate is pretty consistent across the five score situations. The rates ranged from 38.36% when a team is trailing by 9+ to 41.13% when the game is tied. This suggests that score differential by itself may not predict whether a team would successfully convert on third down.

7. Model Selection

7.1 Preparing the Plays for the Model

```
import statsmodels.api as sm #GLMs
import warnings
warnings.filterwarnings('ignore') #identity link warning

# drop a row if yards or goal-to-go is missing
third = third_down_analysis[
    (third_down_analysis['ydstogo'] >= 1)
    & (third_down_analysis['goal_to_go'].notna())
].copy()

third['yards'] = third['ydstogo'].astype(int)
third['converted'] = third['third_down_converted'].astype(int) #1 = converted
third['goal'] = (third['goal_to_go'] == 1).astype(int) #1 = goal-to-go

# scramble is stored as a run, code it as a pass
third['play'] = third['play_type']
third.loc[third['qb_scramble'].fillna(0) == 1, 'play'] = 'pass'
third['run'] = (third['play'] == 'run').astype(int) #1 = called run
third['yards_run'] = third['yards'] * third['run'] #interaction

# 2026 is only weeks 1-3
fit = third[third['season'].isin([2023, 2024, 2025])].copy()
holdout = third[third['season'] == 2026].copy()

n_scramble = int((third['qb_scramble'].fillna(0) == 1).sum())
n_file_runs = int((third['play_type'] == 'run').sum())
print(f'2023-2025 plays: {len(fit):,}')
print(f'Scrambles moved to the pass: {n_scramble:,} (out of {n_file_runs:,} plays the file
called runs)')
display(fit.groupby('play')['converted'].agg(['mean', 'count']))
print(f'2026 left out: {len(holdout):,} plays')
```

```
2023-2025 plays: 20,769
Scrambles moved to the pass: 1,096 (out of 5,493 plays the file called runs)
```

	mean	count
play		
pass	0.361884	16602

	mean	count
play		
run	0.532277	4167

```
2026 left out: 1,179 plays
```

The model is fit on 20,769 plays from the 2023–2025 seasons. Out of 5,493 plays the file called runs, 1,096 were scrambles and were coded as passes. On the plays used in the fit, called runs converted 53.23% of the time compared with 36.19% for passes. The 1,179 plays from the first three weeks of 2026 were left out of the fit.

7.2 Linear Model

```
# OLS on the 0/1 outcome
X = sm.add_constant(fit[['yards', 'run', 'yards_run', 'goal']])
linear_model = sm.OLS(fit['converted'], X).fit()
display(linear_model.summary())
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                            OLS Regression Results
==============================================================================
Dep. Variable:              converted   R-squared:                       0.115
Model:                            OLS   Adj. R-squared:                  0.115
Method:                 Least Squares   F-statistic:                     676.0
Date:                Mon, 19 Oct 2026   Prob (F-statistic):               0.00
Time:                        22:11:59   Log-Likelihood:                -13344.
No. Observations:               20769   AIC:                         2.670e+04
Df Residuals:                   20764   BIC:                         2.674e+04
Df Model:                           4
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.5819      0.007     83.948      0.000       0.568       0.595
yards         -0.0279      0.001    -36.830      0.000      -0.029      -0.026
run            0.1192      0.011     10.558      0.000       0.097       0.141
yards_run     -0.0127      0.002     -8.159      0.000      -0.016      -0.010
goal          -0.0908      0.014     -6.473      0.000      -0.118      -0.063
==============================================================================
Omnibus:                   116201.644   Durbin-Watson:                   1.977
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             2364.899
Skew:                           0.309   Prob(JB):                         0.00
Kurtosis:                       1.467   Cond. No.                         38.5
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
"""
```

This fits a straight line to a 0 or 1 outcome. The R-squared is 0.115, so yards to go, play type, and goal-to-go leave most of the play-to-play result unexplained. A straight line can also fall below 0 or rise above 1, which is not a usable chance.

7.3 Identity-Link Model

```
# binomial GLM, identity link
identity_model = sm.GLM(
    fit['converted'],
    X,
    family=sm.families.Binomial(link=sm.families.links.Identity()),
).fit()
display(identity_model.summary())
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
```

```
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:              converted   No. Observations:                20769
Model:                            GLM   Df Residuals:                    20764
Model Family:                Binomial   Df Model:                            4
Link Function:               Identity   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                    nan
Date:                Mon, 19 Oct 2026   Deviance:                       26126.
Time:                        22:11:59   Pearson chi2:                 1.95e+04
No. Iterations:                    10   Pseudo R-squ. (CS):                nan
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.4611      0.005     97.970      0.000       0.452       0.470
yards         -0.0115      0.000    -97.970      0.000      -0.012      -0.011
run            0.1662      0.010     17.298      0.000       0.147       0.185
yards_run     -0.0059      0.000    -22.591      0.000      -0.006      -0.005
goal          -0.0754      0.014     -5.303      0.000      -0.103      -0.048
==============================================================================
"""
```

The identity-link model is still a straight line, but it is fit as a binomial model. It stays closer to 0 and 1 than the linear fit. It still misses the observed rates at short distances and at 20 yards, so it is not the model used for the rest of the notebook.

7.4 Logistic Model

```
# binomial GLM, logit link
logit_model = sm.GLM(fit['converted'], X, family=sm.families.Binomial()).fit()
display(logit_model.summary())
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:              converted   No. Observations:                20769
Model:                            GLM   Df Residuals:                    20764
Model Family:                Binomial   Df Model:                            4
Link Function:                  Logit   Scale:                          1.0000
Method:                          IRLS   Log-Likelihood:                -12522.
Date:                Mon, 19 Oct 2026   Deviance:                       25045.
Time:                        22:11:59   Pearson chi2:                 2.11e+04
No. Iterations:                     6   Pseudo R-squ. (CS):             0.1279
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.5384      0.035     15.550      0.000       0.471       0.606
yards         -0.1504      0.004    -33.854      0.000      -0.159      -0.142
run            0.6831      0.062     11.004      0.000       0.561       0.805
yards_run     -0.1823      0.016    -11.366      0.000      -0.214      -0.151
goal          -0.4548      0.067     -6.811      0.000      -0.586      -0.324
==============================================================================
"""
```

The logistic model keeps every fitted chance between 0 and 1. A longer distance lowers the log-odds, and a goal-to-go situation lowers them as well. This is the model used from here on.

7.5 Comparing the Models at 1, 10, and 20 Yards

```
# const, yards, run, yards_run, goal
def fitted_chance(model, yards, play_name, goal_flag):
    run = 1 if play_name == 'run' else 0
    return float(model.predict([[1, yards, run, yards * run, goal_flag]])[0])
```

```
rows = []
for yards in [1, 10, 20]:
    for play_name in ['run', 'pass']:
        these = fit[(fit['yards'] == yards) & (fit['play'] == play_name) & (fit['goal'] == 0)]
        rows.append({
            'yards': yards,
            'play': play_name,
            'n': len(these),
            'observed': these['converted'].mean(),
            'linear': fitted_chance(linear_model, yards, play_name, 0),
            'identity': fitted_chance(identity_model, yards, play_name, 0),
            'logit': fitted_chance(logit_model, yards, play_name, 0),
        })
comparison = pd.DataFrame(rows)
display(comparison.round(3))

# fitted values outside 0 to 1
for name, model in [('linear', linear_model), ('identity', identity_model), ('logit',
logit_model)]:
    fitted = model.predict(X)
    n_outside = int(((fitted < 0) | (fitted > 1)).sum())
    print(f'{name}: {n_outside:,} outside 0 to 1, range {fitted.min():.3f} to
{fitted.max():.3f}')

display(round(linear_model.rsquared, 3))
display(round(logit_model.aic, 1))
display(identity_model.aic) #nan if a fit hits 0 or 1
```

	yards	play	n	observed	linear	identity	logit
0	1	run	1715	0.741	0.660	0.610	0.709
1	1	pass	561	0.583	0.554	0.450	0.596
2	10	run	83	0.084	0.295	0.453	0.109
3	10	pass	1591	0.280	0.303	0.346	0.276
4	20	run	23	0.000	−0.111	0.279	0.004
5	20	pass	147	0.027	0.024	0.231	0.078

```
linear: 501 outside 0 to 1, range −0.761 to 0.660
identity: 1 outside 0 to 1, range −0.000 to 0.610
logit: 0 outside 0 to 1, range 0.000 to 0.709
```

```
np.float64(0.115)
```

```
np.float64(25055.0)
```

```
np.float64(nan)
```

At 1 yard the logistic chances, 0.709 for a called run and 0.596 for a pass, are the closest to the observed rates of 0.741 and 0.583. The linear fit falls outside 0 to 1 on 501 plays, and at 20 yards a called run is fit at −0.111. The identity fit is still about 0.279 and 0.231 at 20 yards, where the observed rates are 0.000 and 0.027. The logistic model follows the observed rates and stays inside 0 to 1, so it is the one that is kept.

8. Logistic Regression

8.1 Odds Ratios

```
# odds ratio
display(logit_model.params.round(4))
display(np.exp(logit_model.params).round(4))
display(np.exp(logit_model.conf_int()).round(4))
```

```
const        0.5384
yards       −0.1504
run          0.6831
yards_run   −0.1823
goal        −0.4548
dtype: float64
```

```
const        1.7132
yards        0.8604
run          1.9800
yards_run    0.8333
goal         0.6346
dtype: float64
```

	0	1
const	1.6008	1.8335
yards	0.8529	0.8679
run	1.7532	2.2362
yards_run	0.8075	0.8600
goal	0.5567	0.7233

Each additional yard multiplies the odds of a conversion by 0.8604, with a 95% confidence interval from 0.8529 to 0.8679. Goal-to-go multiplies the odds by 0.6346, with an interval from 0.5567 to 0.7233. Both intervals sit below 1, so both lower the odds of converting.

8.2 Pass Line and Called-Run Line

```
# pass line is intercept + yards slope
# called-run line adds the run term and the interaction
b0 = logit_model.params['const']
b_yards = logit_model.params['yards']
b_run = logit_model.params['run']
b_int = logit_model.params['yards_run']

pass_intercept = b0
pass_slope = b_yards
run_intercept = b0 + b_run
run_slope = b_yards + b_int
crossover = -b_run / b_int #yards where the lines meet

print('Pass line:', round(pass_intercept, 3), '+', round(pass_slope, 3), '* yards')
print('Called-run intercept:', round(run_intercept, 3))
print('Called-run slope:', round(run_slope, 3))
print('Lines cross at', round(crossover, 1), 'yards when it is not goal-to-go')
print('Goal-to-go multiplies the odds by', round(np.exp(logit_model.params['goal']), 3))
```

```
Pass line: 0.538 + -0.15 * yards
Called-run intercept: 1.222
Called-run slope: -0.333
Lines cross at 3.7 yards when it is not goal-to-go
Goal-to-go multiplies the odds by 0.635
```

For a pass, the log-odds start at 0.538 and fall by 0.150 for each extra yard. A called run starts higher, at 1.222, and falls faster, by 0.333 per yard. The two lines meet at 3.7 yards when it is not goal-to-go, and past that point the pass has the higher chance. Goal-to-go multiplies the odds by 0.635 and does not change either slope.

9. Model Analysis

9.1 Fitted Conversion Probabilities

```
rows = []
for yards in [1, 2, 3, 4, 6, 8, 10, 15]:
    for play_name in ['run', 'pass']:
        for goal_flag, situation in [(0, 'open field'), (1, 'goal line')]:
            rows.append([
                yards,
                situation,
```

```
                play_name,
                fitted_chance(logit_model, yards, play_name, goal_flag),
            ])
use_table = pd.DataFrame(rows, columns=['yards_to_go', 'situation', 'play', 'fitted_chance'])
display(
    use_table.pivot_table(
        index=['situation', 'yards_to_go'],
        columns='play',
        values='fitted_chance',
    ).round(3)
)
```

play		pass	run
situation	yards_to_go		
goal line	1	0.483	0.607
	2	0.446	0.525
	3	0.409	0.442
	4	0.373	0.363
	6	0.306	0.226
	8	0.246	0.131
	10	0.195	0.072
	15	0.102	0.014
open field	1	0.596	0.709
	2	0.559	0.636
	3	0.522	0.556
	4	0.484	0.473
	6	0.410	0.315
	8	0.340	0.192
	10	0.276	0.109
	15	0.152	0.023

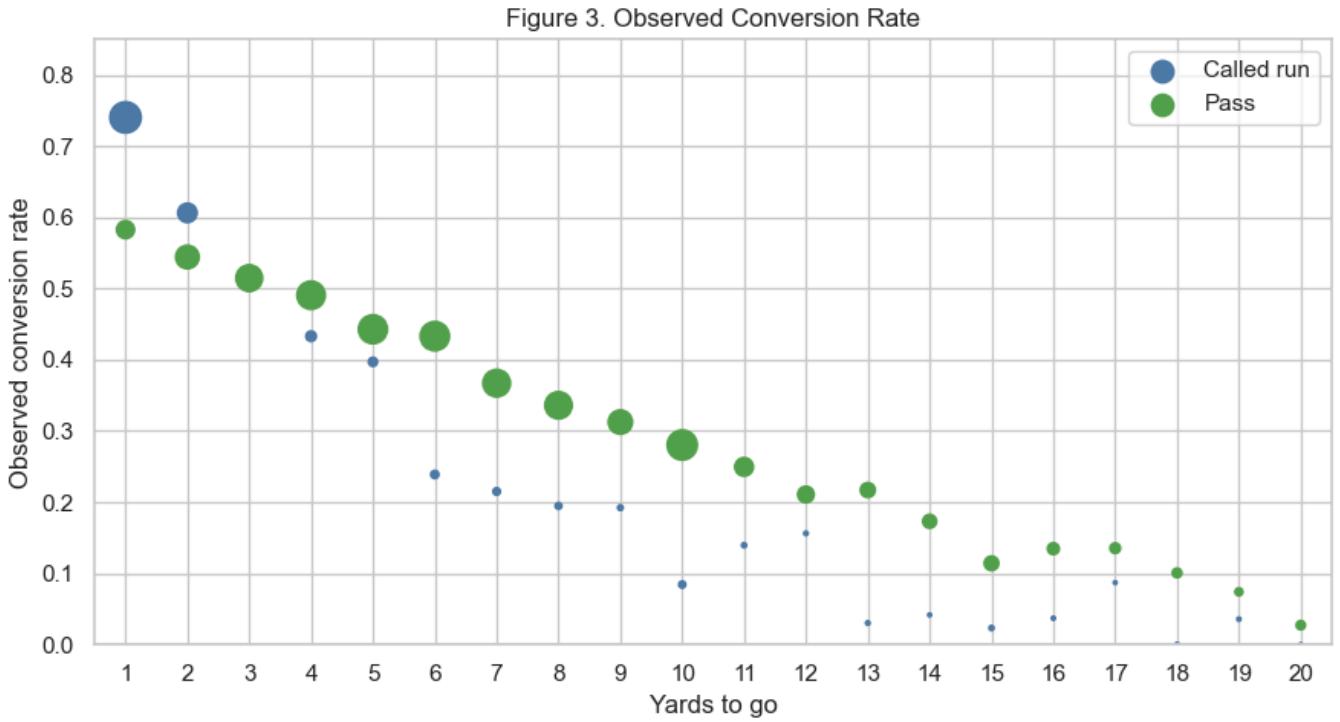
At 1 yard a called run has the higher fitted chance, 0.709 compared with 0.596 for a pass. By 4 yards the pass is already slightly higher, 0.484 against 0.473, and at 10 yards the gap is 0.276 for a pass against 0.109 for a called run. On the goal line both chances fall, and a 3rd-and-1 called run drops from 0.709 to 0.607. This doesn't mean a team would convert at these rates if the other call were required. These are the chances for the calls that were actually used.

9.2 Observed Open-Field Rates

```
# figure 3, open field, yards 1 to 20
open_field = fit[fit['goal'] == 0]
observed_rows = []
for (yards, play_name), group in open_field.groupby(['yards', 'play']):
    observed_rows.append({
        'yards': int(yards),
        'play': play_name,
        'n': len(group),
        'rate': float(group['converted'].mean()),
    })
observed = pd.DataFrame(observed_rows)
observed = observed[observed['yards'] <= 20]

plt.figure(figsize=(9, 5))
for play_name, color, label in [('run', '#4c78a8', 'Called run'), ('pass', '#54a24b', 'Pass')]:
    group = observed[observed['play'] == play_name]
    plt.scatter(group['yards'], group['rate'], s=group['n'] / 8, color=color, label=label) #dot
size follows n
plt.xticks(range(1, 21, 1))
plt.xlim(0.5, 20.5)
plt.ylim(0, 0.85)
plt.xlabel('Yards to go')
plt.ylabel('Observed conversion rate')
plt.title('Figure 3. Observed Conversion Rate')
plt.legend()
```

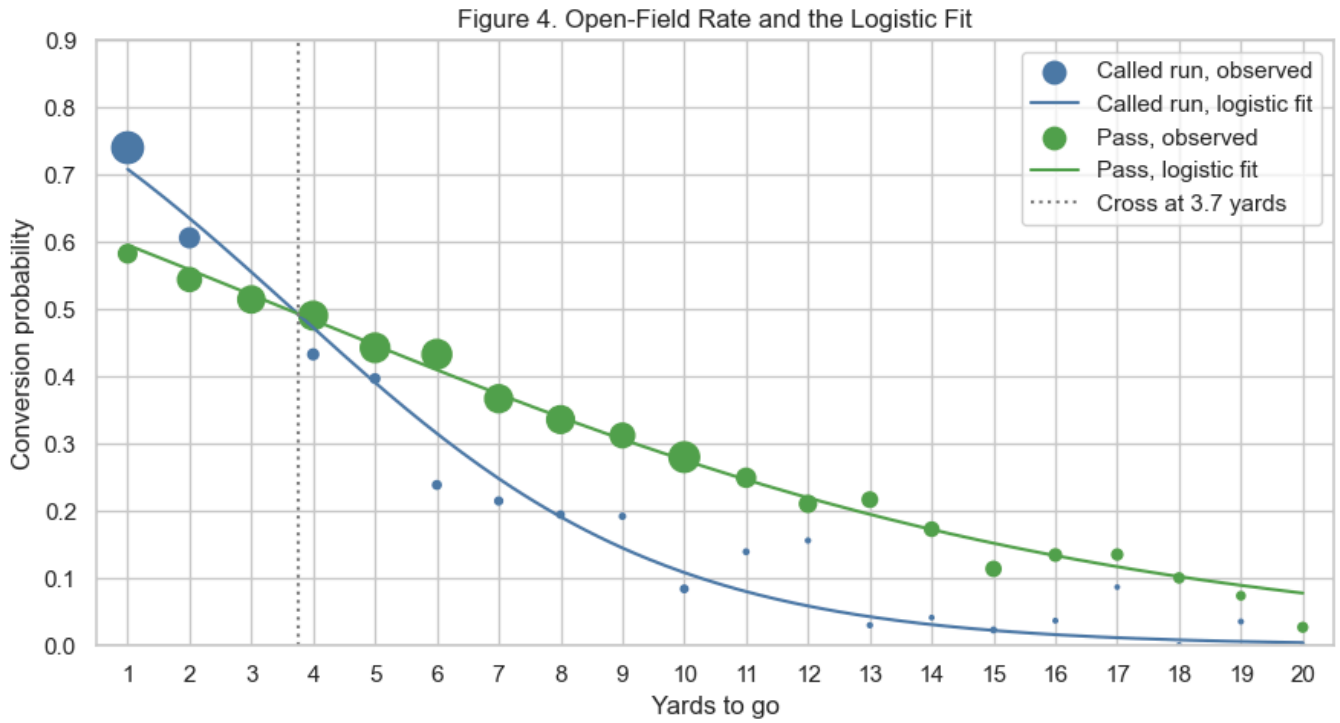
```
plt.tight_layout()
plt.show()
```



This plot shows the observed open-field conversion rate from 1 to 20 yards. Called runs convert more often when few yards are needed, and passes convert more often once the distance gets longer. The larger dots are the distances with more plays.

9.3 Open-Field Rate and the Logistic Fit

```
# figure 4, open field with the logit
yard_grid = np.linspace(1, 20, 191)
plt.figure(figsize=(9, 5))
for play_name, color, label in [('run', '#4c78a8', 'Called run'), ('pass', '#54a24b', 'Pass')]:
    group = observed[observed['play'] == play_name]
    plt.scatter(group['yards'], group['rate'], s=group['n'] / 8, color=color, label=f'{label},
observed')
    curve = [fitted_chance(logit_model, float(yard), play_name, 0) for yard in yard_grid] #goal
= 0
    plt.plot(yard_grid, curve, color=color, label=f'{label}, logistic fit')
plt.axvline(crossover, color='gray', linestyle=':', label=f'Cross at {crossover:.1f} yards')
plt.xticks(range(1, 21, 1))
plt.xlim(0.5, 20.5)
plt.ylim(0, 0.9)
plt.xlabel('Yards to go')
plt.ylabel('Conversion probability')
plt.title('Figure 4. Open-Field Rate and the Logistic Fit')
plt.legend()
plt.tight_layout()
plt.show()
```



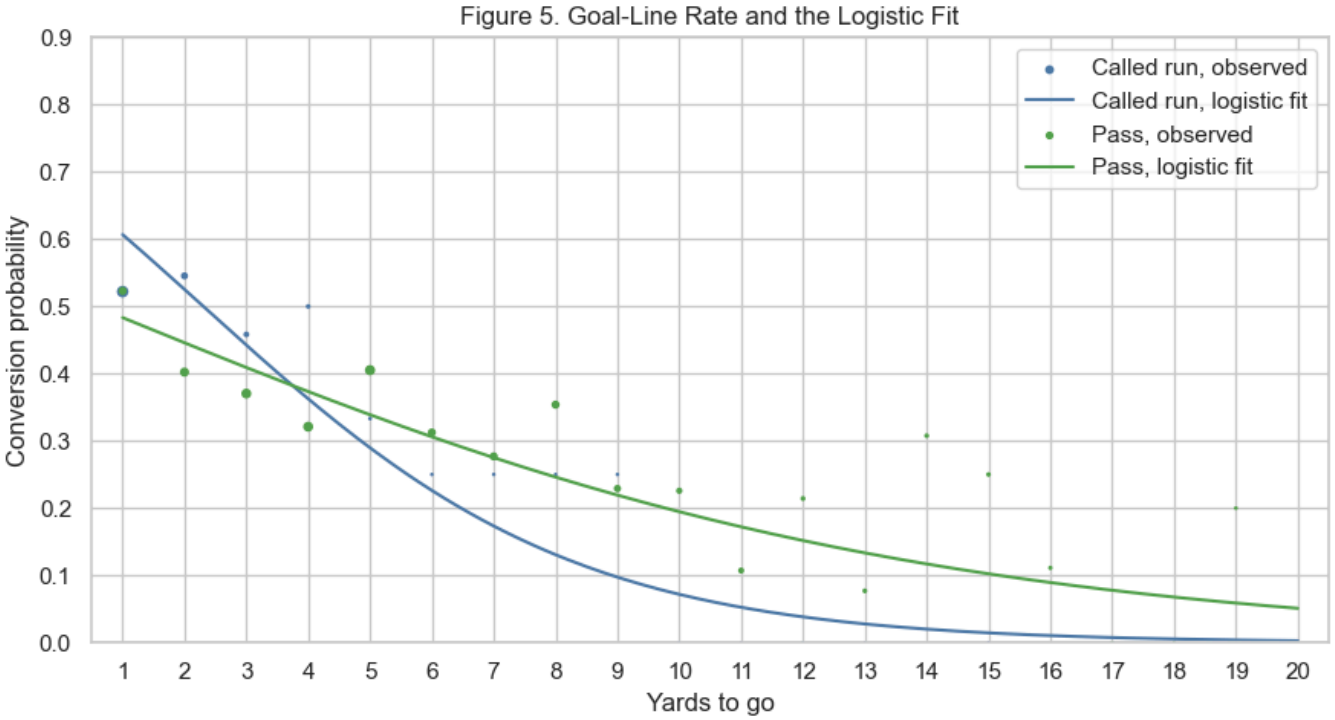
The curves are the logistic fit on the same open-field plays. They cross at 3.7 yards, which is where the pass chance overtakes the called-run chance. The short called-run points sit a little above the curve, while the pass points stay close to it.

9.4 Goal-Line Rate and the Logistic Fit

```
# figure 5, goal line
goal_line = fit[fit['goal'] == 1]
goal_rows = []
for (yards, play_name), group in goal_line.groupby(['yards', 'play']):
    goal_rows.append({
        'yards': int(yards),
        'play': play_name,
        'n': len(group),
        'rate': float(group['converted'].mean()),
    })
goal_observed = pd.DataFrame(goal_rows)
goal_observed = goal_observed[goal_observed['yards'] <= 20]

plt.figure(figsize=(9, 5))
for play_name, color, label in [('run', '#4c78a8', 'Called run'), ('pass', '#54a24b', 'Pass')]:
    group = goal_observed[goal_observed['play'] == play_name]
    plt.scatter(group['yards'], group['rate'], s=group['n'] / 8, color=color, label=f'{label},
observed')
    curve = [fitted_chance(logit_model, float(yard), play_name, 1) for yard in yard_grid] #goal
= 1
    plt.plot(yard_grid, curve, color=color, label=f'{label}, logistic fit')
plt.xticks(range(1, 21, 1))
plt.xlim(0.5, 20.5)
plt.ylim(0, 0.9)
plt.xlabel('Yards to go')
plt.ylabel('Conversion probability')
plt.title('Figure 5. Goal-Line Rate and the Logistic Fit')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



Goal-to-go shifts both curves down. A called run is still higher at 1 yard, and a pass is higher once the distance gets past about 4 yards. There are fewer goal-line plays, so the points jump around more than they do in the open field.

9.5 2026 Plays Left Out of the Fit

```
# 2026 plays not used in the fit
Xh = sm.add_constant(holdout[['yards', 'run', 'yards_run', 'goal']])
holdout = holdout.copy()
holdout['predicted'] = logit_model.predict(Xh)
print(
    f"2026 left out of the fit: actual {holdout['converted'].mean():.3f}, "
    f"model says {holdout['predicted'].mean():.3f}, n = {len(holdout):,}"
)
```

```
2026 left out of the fit: actual 0.412, model says 0.393, n = 1,179
```

These 1,179 plays from the first three weeks of 2026 were not used to fit the line. The model expected a conversion rate of 0.393, and the plays converted at 0.412. The sample is only three weeks, so the difference should be interpreted cautiously and the line was not refit.